02강

프로그래밍 언어론

프로그래밍 언어의 발전 및 동작 원리

컴퓨터과학과 김진욱교수

학습목차



1. 프로그래밍 언어의 발전
2. 프로그램 동작 원리
3. 프로그래밍 언어의 평가 기준

01

프로그래밍 언어의 발전

1. 프로그래밍 언어의 발전

- 컴퓨터 시스템의 발전
  - 아이디어 시대
    - 계산 자동화를 위한 상상 속의 기계 설계(튜링기계)
  - 전자식 컴퓨터 등장
    - 전자 신호를 통한 계산 기계(에니악, 콜로서스)
  - 프로그램 저장 방식의 컴퓨터
    - 프로그램과 처리기를 분리(에드박)

1. 프로그래밍 언어의 발전

- 운영체제의 발전
  - 일괄처리 운영체제
    - 관리자(operator)를 대신할 프로그램 등장
  - 시분할 운영체제
    - 한 컴퓨터를 여러 사람이 동시에 사용
  - DOS
    - IBM 컴퓨터, Apple 등 개인용 컴퓨터(PC) 등장
  - PC 환경 운영체제 발전
    - GUI 운영체제와 Linux의 발전

1. 프로그래밍 언어의 발전

1950년대: 초기 프로그래밍 언어

Fortran: 수식과 문장, 제어문의 등장

- IBM의 존 배커스(John Backus)가 개발
- 과학계산용 언어(formula translation)

```
      SMALLA = A(1)
5     DO 20 I = 2, 10
      IF (A(I) - SMALLA) 10, 20, 20
10    SMALLA = A(I)
20    CONTINUE
      PRINT 2, SMALLA
2     FORMAT ("MIN IS" F7.2)
      STOP 77777
```

1. 프로그래밍 언어의 발전

1950년대: 초기 프로그래밍 언어

Algol: 구조화 프로그래밍의 발전

- 본래 이름: IAL(International Algebraic Language)
- 국제 위원회 ACM-GAMM을 통해 설계된 언어
- 알고리즘 기술 언어

```
if (m > a);
  begin
    m = a;
    found = 1
  end;
```

1. 프로그래밍 언어의 발전

1950년대: 초기 프로그래밍 언어

LISP: 초기 함수형 언어

- MIT의 존 매카시(John McCarthy)가 설계함
- 최초의 함수형 언어

```
(+ 3 (* 2 1))
```

1. 프로그래밍 언어의 발전

1960년대: 프로그래밍 언어의 발전

Cobol: 레코드 타입의 소개

- 미 해군에서 그레이스 호퍼(Grace Hopper)가 이끄는 팀이 개발함
- 사무용 언어

```
01  EMPLOYEE-RECORD.
    05  NAME.
        10 LAST           PIC X(20).
        10 FIRST          PIC X(20).
    05  EMPLOYEE-NUMBER   PIC 9(10).
    05  HOURS-WORKED      PIC 99.
    05  HOUR-RATE         PIC 999V99.
```

1. 프로그래밍 언어의 발전

1960년대: 프로그래밍 언어의 발전

PL/I: 너무 복잡한 언어

- 모든 언어를 합쳐 보았으나 결과적으로 너무 복잡해짐

BASIC: 교육용 언어

- Beginner's All-purpose Symbolic Instruction Code

Simula: 객체지향의 등장

- 시뮬레이션 언어

1. 프로그래밍 언어의 발전

1970년대: 프로그래밍 언어의 단순화

Pascal: 차세대 교육용 언어

- 니클라우스 버트(Niklaus Wirth)가 개발함
- 구조화 프로그래밍 지원

C: 진정한 시스템 프로그래밍 언어

- 데니스 리치(Dennis Ritchie)가 개발함
- Unix 개발용 시스템 프로그래밍 언어
- Objective-C, C++, Java, C# 등 다양한 언어에 지대한 영향을 끼침

1. 프로그래밍 언어의 발전

1970년대: 프로그래밍 언어의 단순화

Prolog: 선언적 논리 언어

- 최초의 논리 언어

Smalltalk: 객체지향 언어의 발전

- 최초로 GUI, 마우스 등을 도입

```
3 factorial + 5 factorial
```

1. 프로그래밍 언어의 발전

1970년대: 프로그래밍 언어의 단순화

Ada: 안전성을 위한 대장정

- 미 국방성(DoD) 공모
- 매우 복잡한 언어로 1983년에 첫 컴파일러 등장

ML: 타입 시스템을 갖춘 현대 프로그래밍 언어

- Meta Language의 약자
- 강력한 정적 타입 검사, 타입 추론, 패턴 검사, 예외 처리 등

Scheme: 간결한 LISP

- MIT 학생들의 기초 프로그래밍 언어

1. 프로그래밍 언어의 발전

1980년대: 현대 프로그래밍 언어 등장

Common Lisp: 방대한 LISP의 통합

- 함수형 언어 패러다임과 객체지향 패러다임을 동시에 지원

Objective-C: C의 탈을 쓴 Smalltalk

- C를 기초로 한 객체지향 언어의 신호탄
- Apple의 애플리케이션 작성 언어로 발전

1. 프로그래밍 언어의 발전

1980년대: 현대 프로그래밍 언어 등장

C++: 객체지향으로 변신한 C

- 비얀 스트로스트룹(Bjarne Stroustrup)이 발표함
- 클래스 개념을 C에 도입

Perl: 문자열 처리를 위한 언어

- 래리 월(Larry Wall)이 설계함
- 정규식을 바탕으로 한 강력한 패턴 매칭 기능 포함

1. 프로그래밍 언어의 발전

1990년대 이후: 프로그래밍 언어의 대중화

Java: 단순한 객체지향 언어

- 제임스 고슬링(James Gosling)이 이끌었던 개발팀이 개발
- 임베디드 컴퓨팅 → 웹 브라우저
- JVM(Java Virtual Machine)

JavaScript: 웹 프로그래밍 언어

- Netscape
- Elm, TypeScript 등 다양한 변종 언어로 발전

1. 프로그래밍 언어의 발전

1990년대 이후: 프로그래밍 언어의 대중화

Python: 빠른 프로토타이핑 언어

- 귀도 반 로섬(Guido van Rossum)이 만듦
- 스크립트 언어
- 동적 언어를 추구
- 다중 패러다임 언어

Haskell: 순수 함수형 언어

- 국제위원회가 만든 함수형 언어
- 모나드(monad)가 탑재되면서 점차 인기를 끎
- Scala에 영향을 줌

02

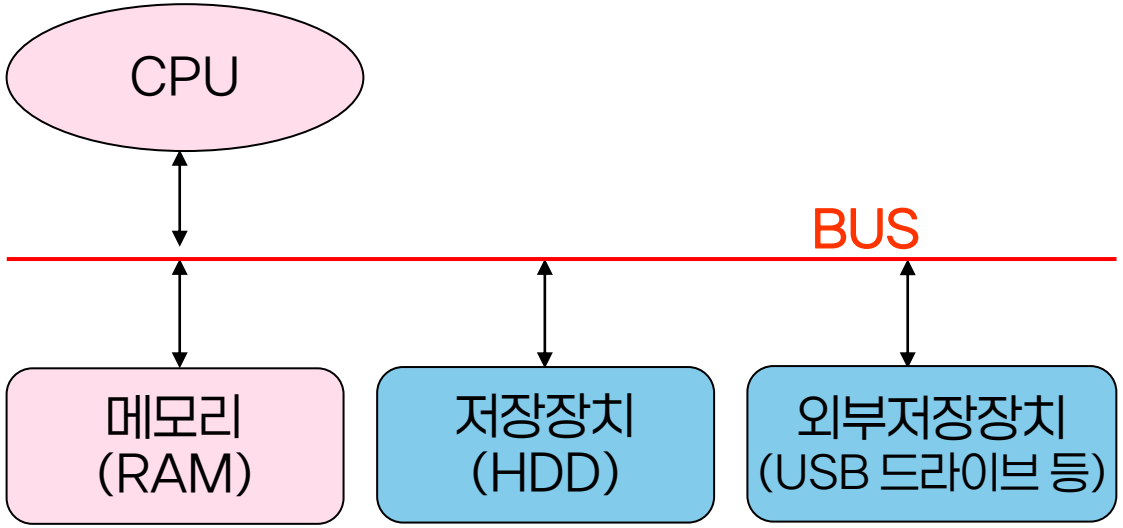
프로그램 동작 원리

2. 프로그램 동작 원리

컴퓨터 구조와 프로그램 동작 원리

컴퓨터 구조

- CPU와 메모리, 저장장치 등이 BUS로 연결
- 다양한 입출력 장치도 BUS에 연결될 수 있음

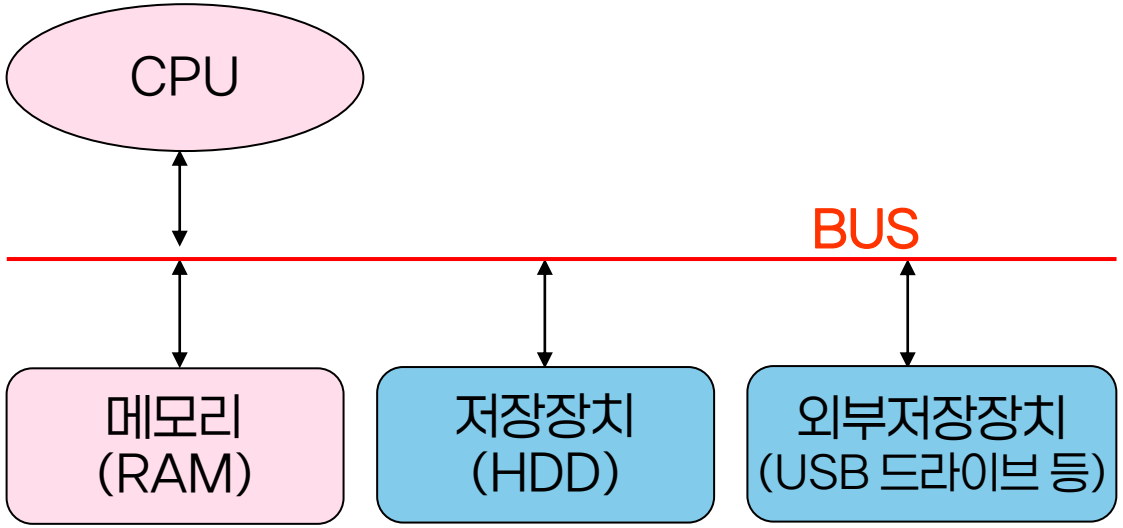


2. 프로그램 동작 원리

❑ 컴퓨터 구조와 프로그램 동작 원리

컴퓨터 동작 원리

- 전원을 켜면 ROM에 있는 BIOS는 하드웨어 점검 후 저장장치의 특정 코드(부트로더)를 수행
- 부트로더는 운영체제를 메모리로 적재하여 수행

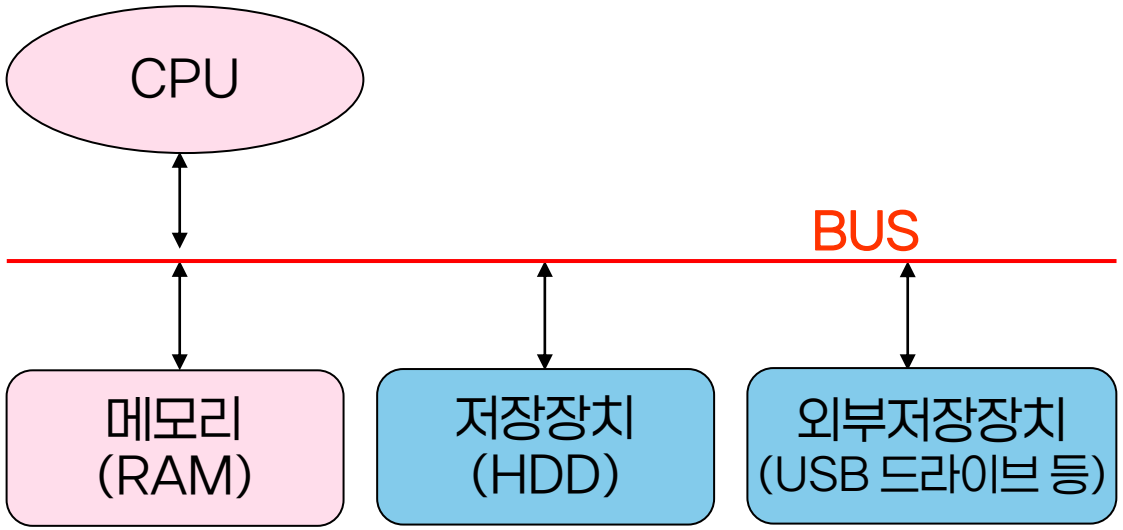


2. 프로그램 동작 원리

컴퓨터 구조와 프로그램 동작 원리

프로그램 동작 원리

- CPU는 인출-해석-실행 주기를 반복하여 메모리에 적재된 프로그램의 명령어를 수행



2. 프로그램 동작 원리

프로그래밍 언어 구현이 필요한 이유

기계어

- CPU가 이해하고 수행하는 명령어
- 이진수 형태의 명령어를 사람이 이해하는 것은 매우 난해

어셈블리어

- 기계어에 거의 일대일 대응하는 형태의 기호 언어
- CPU에 종속적 → 이식성이 거의 없음

2. 프로그램 동작 원리

❑ 프로그래밍 언어 구현이 필요한 이유

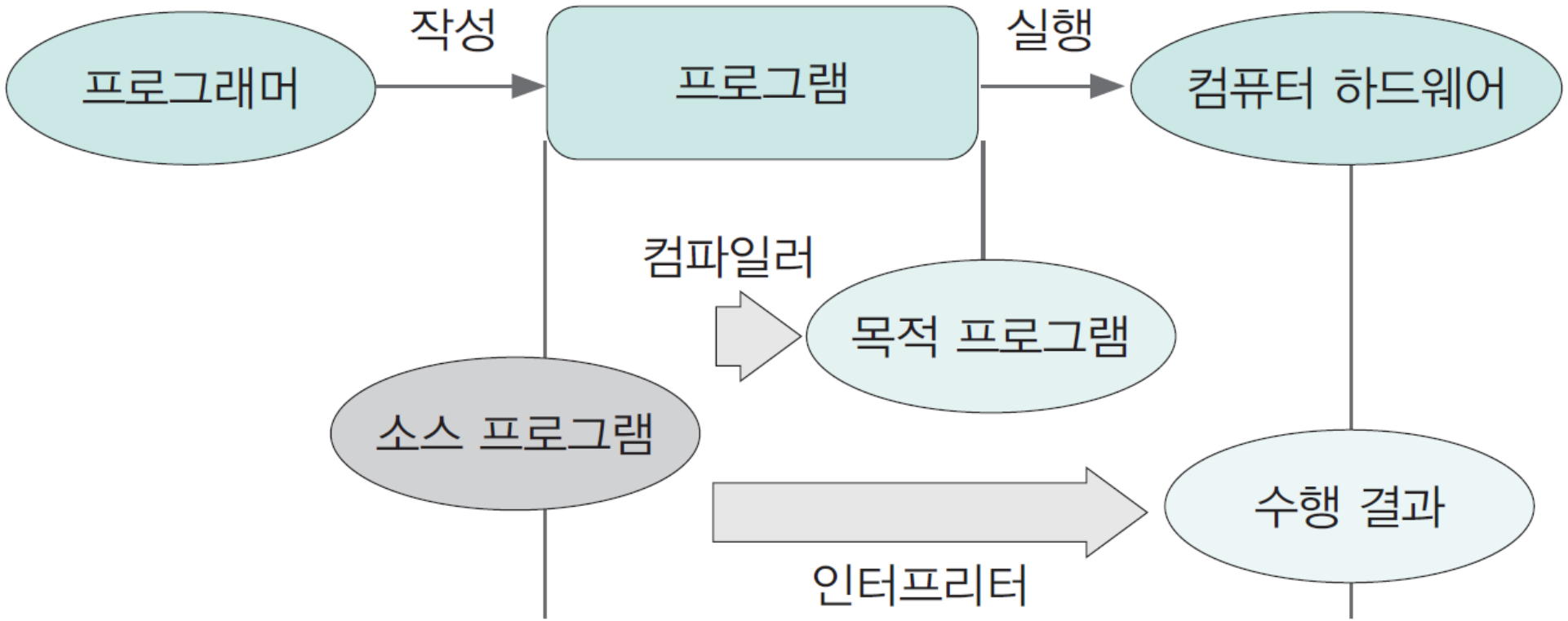
고급 프로그래밍 언어

- 사람에 가까운 표현으로 프로그램을 나타냄
- 특정 기계에 종속적이지 않음
- 프로그램을 CPU가 알아듣는 기계어로 표현해 주어야 함

- 소스 프로그램과 목적 프로그램의 간극을 메꾸어야 함

2. 프로그램 동작 원리

프로그래밍 언어의 구현 방법

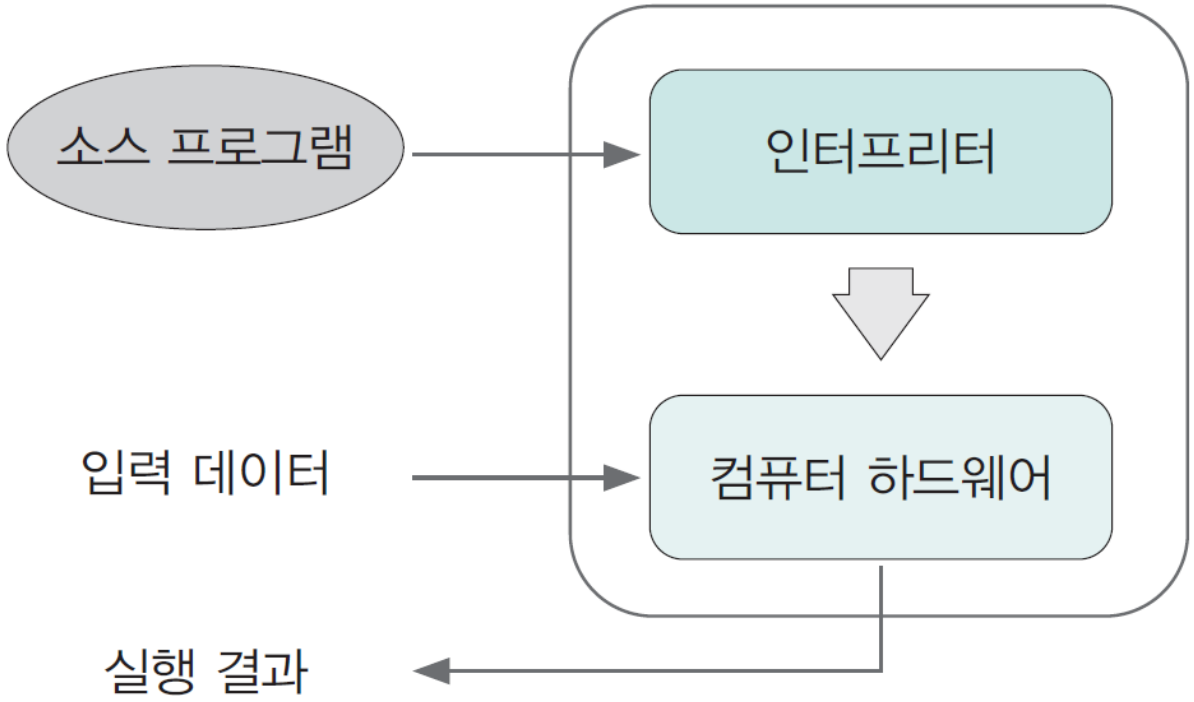


2. 프로그램 동작 원리

프로그래밍 언어의 구현 방법

인터프리터

- 프로그래밍 언어로 작성된 고수준의 명령을 해석하여 수행하는 프로그램
- 인터프리터는 CPU의 인출-해석-실행 주기를 흉내 냄

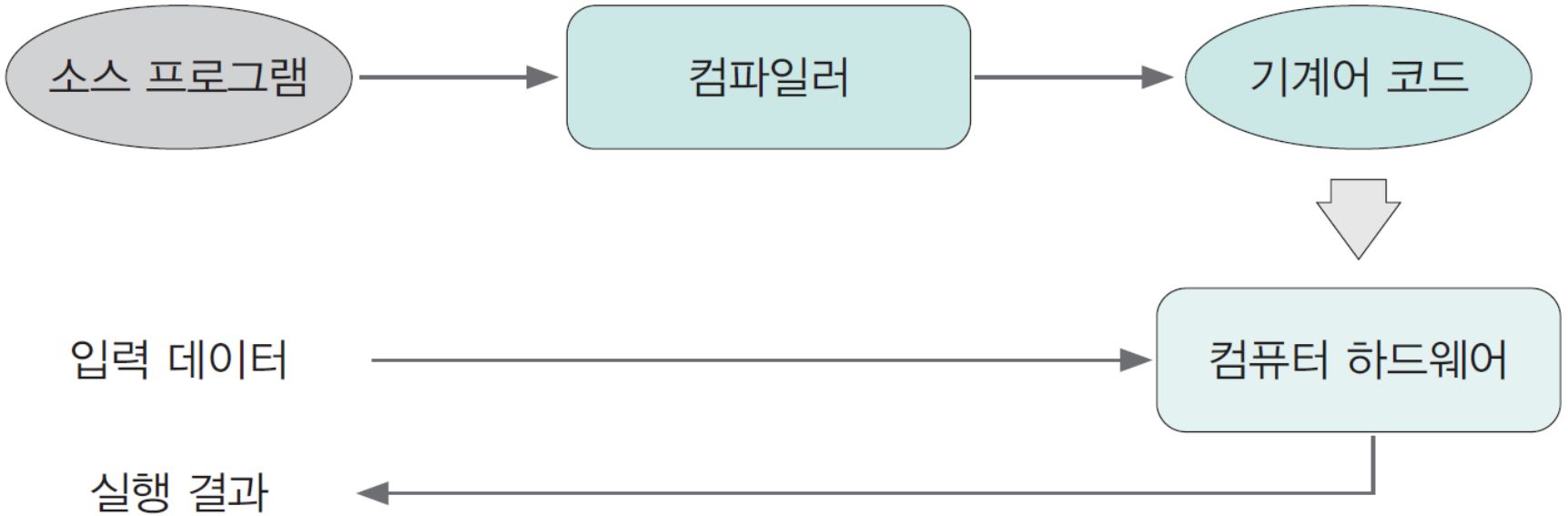


2. 프로그램 동작 원리

프로그래밍 언어의 구현 방법

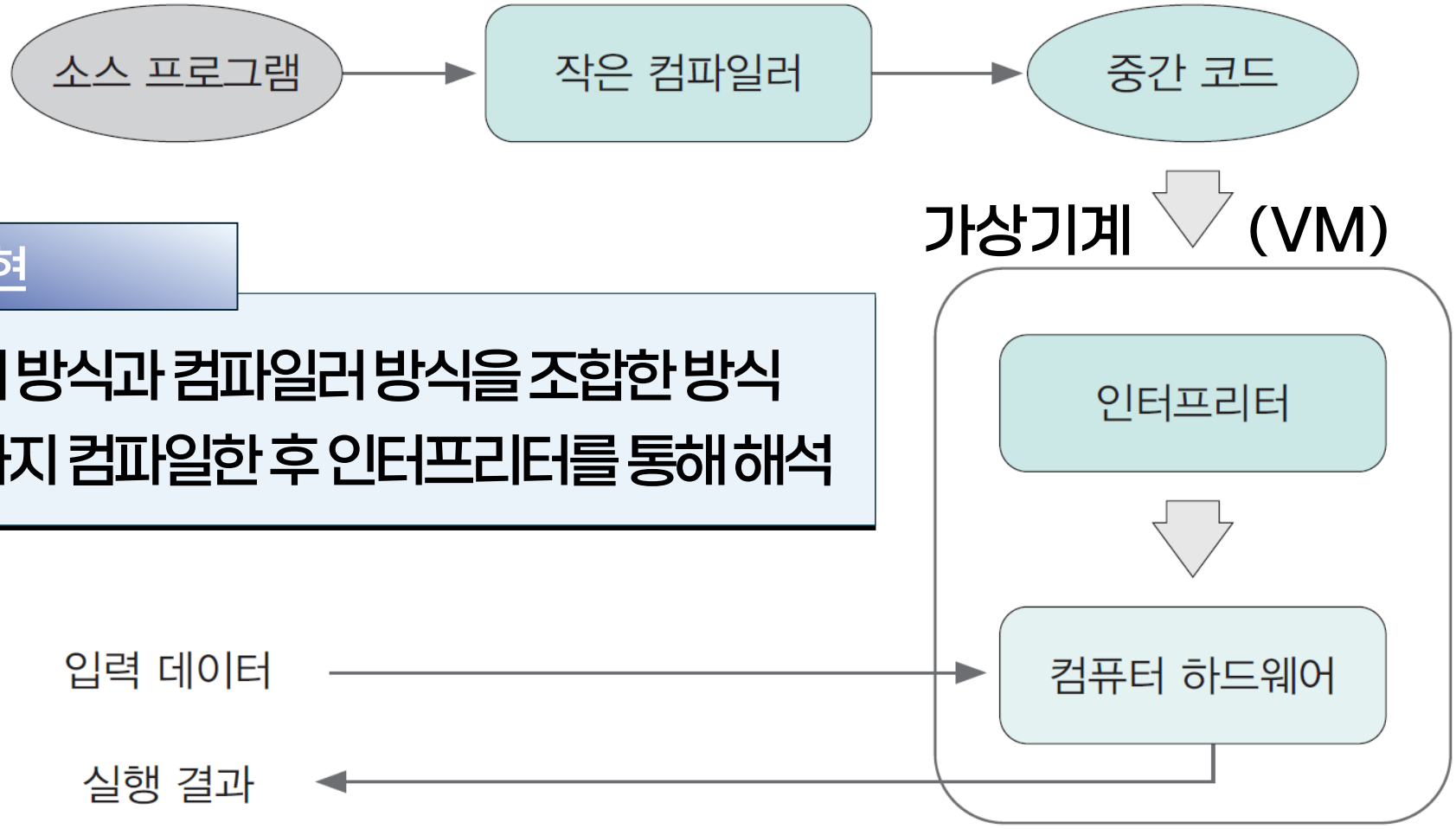
컴파일러

- 프로그램을 CPU가 수행할 수 있는 형태로 바꾸어서 CPU가 실행
- 인터프리터가 하는 해석 과정을 미리 모두 수행(효율적)
- 상용 프로그램은 컴파일 방식을 통해 번역된 후 판매



2. 프로그램 동작 원리

프로그래밍 언어의 구현 방법



하이브리드 구현

- 인터프리터 방식과 컴파일러 방식을 조합한 방식
- 중간코드까지 컴파일한 후 인터프리터를 통해 해석

03

프로그래밍 언어의 평가 기준

3. 프로그래밍 언어의 평가 기준

- 프로그래밍 언어의 요구사항
  - 표현 풍부성(expressiveness)
    - 프로그래머의 아이디어를 표현할 수 있어야 함
  - 유지 보수성(maintainability)
    - 변화에 쉽게 대처할 수 있어야 함
  - 실행 가능성(executability)
    - 컴퓨터에서 실행할 수 있어야 함

3. 프로그래밍 언어의 평가 기준

프로그래밍 언어 설계 원칙

- 규칙성(regularity)
  - 언어의 기능이 잘 조합될 수 있어야 함

일반성, 직교성, 일관성

- 추상화 지원(support of abstraction)
  - 실세계의 대상을 추상화하여 나타낼 수 있고 이를 대상으로 어떤 연산을 수행할 수 있어야 함

데이터 추상화,
제어 추상화,
추상 데이터 타입 정의

- 복잡도 제어(complexity control)
  - 복잡한 대상 및 처리 방법을 제어할 수 있어야 함

캡슐화, 모듈화

3. 프로그래밍 언어의 평가 기준

평가 기준

- 작성력: 프로그램 수식이나 문장, 기능을 쉽게 표현할 수 있는가
- 가독성: 작성된 프로그램을 보고 쉽게 이해할 수 있도록 하는가
- 신뢰성: 작성된 프로그램이 오류에 빠지는 가능성을 줄이는가
- 직교성: 언어 기능이 서로 간섭하지 않고 자유롭게 조합될 수 있는가
- 일관성: 유사한 기능을 같은 형태로 나타낼 수 있는가
- 확장성: 사용자가 원하는 새로운 기능을 추가할 수 있는가
- 효율성: 작성된 프로그램이 효율적으로 수행될 수 있도록 하는가
- 유연성: 프로그래머가 표현하고 싶은 내용을 유연하게 수용하는가
- 이식성: 프로그램을 다른 실행 환경으로 이전할 수 있는가

3. 프로그래밍 언어의 평가 기준

요구사항 / 평가 기준	표현 풍부성	유지 보수성	실행 가능성
작성력	O	O	
가독성		O	
신뢰성		O	O
직교성	O	O	
일관성	O	O	
확장성	O	O	
효율성		O	O
유연성	O	O	
이식성		O	O

3. 프로그래밍 언어의 평가 기준

평가 기준 사이의 절충

프로그램 사용자 측면		프로그램 개발자 측면	
검사 비용을 줄여야 함	효율성 ⟷	신뢰성	더 많이 검사해야 함
간단한 기능이 이해하기 쉬움	가독성 ⟷	작성력	복잡하더라도 많은 기능이 지원되어야 함
안전을 위해 더 많은 제약을 가해야 함	신뢰성 ⟷	유연성	프로그램 작성 시 제약이 적어야 함

3. 프로그래밍 언어의 평가 기준

프로그래밍 언어의 선택 기준

- 해당 프로그래밍 언어를 사용하는 커뮤니티가 활발하고 호의적인 언어
- 특정 응용 분야가 존재하는 프로그래밍 언어
- 접해 보지 못한 프로그래밍 패러다임을 지원할 수 있는 프로그래밍 언어

정리하기

- 1950년대 초기 프로그래밍 언어로는 Fortran, LISP, Algol 등이 있고, 1960년대에는 Cobol, PL/I, BASIC, Simula 등이 유명
- 1970년대에는 C, Pascal, Prolog, Smalltalk, Ada, ML, Scheme 등이 발표되고, 1980년대에는 Common Lisp, Objective-C, C++, Perl 등이 등장
- 1990년대에는 Java, JavaScript, Python, Haskell 등이 등장
- 인터프리터는 고급언어 프로그램으로 작성된 문장을 하나씩 읽어서 수행하고, 컴파일러는 기계어로 미리 번역함
- 프로그래밍 언어의 평가 기준은 상충될 수 있기에 적절한 지점에서 절충해야 함

03강

다음시간안내 ▶▶▶

프로그래밍 언어 패러다임